

面向资源约束量子布尔函数综合的神经蒙特卡洛树搜索方法

中文论文稿 v18: ROS-style LUT proxy 版

2026 年 7 月 8 日

摘要

量子布尔函数 oracle 综合把经典谓词嵌入量子算法, 是 Grover 搜索、量子密码分析和约束求解类量子程序中的基础编译步骤。现有路线通常围绕固定布尔表示、可逆逻辑模板或 CNOT 导向覆盖结构展开, 但容错量子计算中的逻辑开销同时受 T-count、CNOT、深度、门数和辅助比特制约。本文将该任务重新表述为代数正规形 (algebraic normal form, ANF) 和固定极性 Reed-Muller (fixed-polarity Reed-Muller, FPRM) 项集合上的资源约束搜索问题, 提出结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构深度策略、depth-frontier policy 和保守 guard 的 Resource-NMCTS 框架。本文方法不依赖 SSHR 的 parallelotope 候选空间; SSHR 仅作为 small-scale CNOT-oriented baseline 参与比较。实验显示, 在 $n \leq 6$ 的 177 个传统函数上, Pareto-Resource-NMCTS 相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%; 相对 weighted ESOP-MILP 获得 167/3/7, 平均 score 降低 29.84%。新增 ROS-style LUT proxy 对 $n = 3-6$ 与 $n = 14, 15, 16, 18$ 共 309 个函数运行 ABC $K = 3, 4, 5$ LUT sweep, 927/927 行均通过 truth-table 检查; best- K proxy 相对固定 $K = 4$ 平均 score 降低 18.12%, 而 Resource-NMCTS 相对该更强 proxy 仍获得 309/0/0, 平均 score 降低 83.77%。在 $n = 18$ 函数级切片上, 完整 Resource-NMCTS 相对 adaptive depth-2 Boolean-ring screen 进一步降低 23.85% 平均 score; 在 $n = 20$ 边界切片上, adaptive depth-2 Boolean-ring screen 相对 AND-direct ANF 降低 11.80% 平均 score。项集级 $n = 20, 22, 24, 28, 32, 36, 40$ scale 实验表明, 结构深度策略可以在接近 all-depth adaptive 质量的同时减少约三分之一时间; depth-frontier policy 在 $n = 20, 28, 40$ 上相对 fixed depth-2 获得 35/0/37, 平均 score 降低 2.19%, 相对 all-depth depth ≤ 4 节省 58.69% 时间。独立 seed 的 $n = 24, 28, 32, 40$ 泛化实验进一步给出 40/0/56、平均 score 降低 1.85%, 且相对 fixed depth-2 无 score loss。所有 4680 个项集级方法行均通过 factor-plan ANF 展开验证和 emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证; 进一步的 $n = 21, 22, 23$ bridge 中, 180/180 个方法行同时通过完整 truth-table oracle 验证、plan 验证和 emitted-circuit 符号验证。新增 emitted-circuit schedule proxy 显示, 在独立 seed 的 $n = 24, 28, 32, 40$ 泛化集中, frontier policy 相对 fixed depth-2 的 T-depth proxy 为 40/0/56、平均降低 1.85%; 在 $n = 21, 22$ 完整 bridge 中为 8/0/4、平均降低 3.32%; 新增 $n = 23$ bridge 中为 4/0/2、平均降低 1.69%, 但相对 fixed depth-2 会增加显式辅助线 lifetime area。本文的阶段性结论是: Resource-NMCTS 已形成一条独立于 SSHR、以低 T-count 和低加权逻辑资源为目标的 Boolean oracle synthesis 主线; 当前最需要补强的是官方 ROS/RevKit/mockturtle 复现、frontier policy 的质量 gap 压缩, 以及真正的后端映射评估。

关键词: 量子 oracle 综合; 布尔函数综合; 强化学习; 蒙特卡洛树搜索; ANF; FPRM; 布尔环; 资源约束编译

作者想法整理与当前论文定位

本稿首先服务于一个清晰的研究主题：基于强化学习与蒙特卡洛树搜索的量子布尔函数综合方法。这个主题的本质不是复现某个已有综合算法，而是把 Boolean oracle synthesis 看作搜索问题：给定布尔函数的 ANF 或 FPRM 项集合，系统需要在大量可计算、可反算、可验证的代数分解动作中选择一条低资源线路生成路径。强化学习、神经排序和 MCTS 的作用，是在候选动作过多时更早找到有资源收益的分解，而不是直接生成不可解释的量子门序列。

因此，本文不应从 SSHR 入手。SSHR 的 parallelootope 结构适合做 small-scale CNOT-oriented synthesis，适合作为重要 baseline，但它不是本文方法的依赖，也不应成为论文主线。更合适的主线是：ANF/FPRM 项集合搜索定义问题，Boolean-ring screen 扩展可用结构，神经动作先验和 MCTS 处理动作组合，depth policy 与 frontier policy 处理搜索预算，guard 保证学习策略不会轻易劣于确定性 baseline。这样的叙事可以自然比较 T-count、CNOT、门数、线路深度和辅助比特，而不是被单个 CNOT 指标锁死。

从投稿角度看，本文当前已有“明显提升”的证据，但证据形态必须如实呈现。小规模传统函数上，本文在低 T-count 和加权 score 上相对 ESOP、weighted ESOP-MILP、direct ANF 和 AND-direct ANF 有稳定优势；高维部分已经从单纯资源估计推进到项集级 plan 验证、emitted-circuit GF(2) 符号验证和 $n = 21, 22, 23$ 完整 truth-table bridge。与此同时，本文还不能声称硬件后端最优，也不能声称 CNOT-only 全面超过 SSHR。当前稿件应把优势写成“资源约束逻辑层搜索”，把边界写成“仍需外部工具链和后端评估”。

下一步最值得投入的方向也很明确。第一，在当前 ROS-style LUT proxy 的基础上继续接入或复现官方 ROS、mockturtle/RevKit 风格 reversible-toolchain，把比较从 proxy 扩展到更标准的外部流程。第二，继续压缩 depth-frontier policy 相对 depth-4 或 all-depth frontier 的 0.61%–0.97% 质量 gap，同时保持 50% 左右的时间节省。第三，在当前 schedule proxy 的基础上进一步做真实后端映射或至少更精细的逻辑层调度，例如 T-depth 模板、辅助量子位生命周期、反算区间和可能的 routing 压力。完成这些以后，论文就更接近“方法贡献”而不是“阶段性项目报告”。

1 引言

给定布尔函数 $f : \{0, 1\}^n \rightarrow \{0, 1\}$ ，量子 oracle 通常实现为

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \quad (1)$$

这一步看似只是把经典逻辑接入量子程序，但在容错量子计算中，它往往决定了非 Clifford 资源和辅助比特压力。若直接按 ANF 单项式逐项实现，线路语义透明，但容易重复计算相似结构；若只优化 CNOT 或只压缩某一种图表示，又可能把成本转移到 T-count、深度或辅助比特上。因此，本文关注的问题不是“如何复现某个已有 CNOT 优化算法”，而是“如何把 Boolean oracle synthesis 写成一个可由 AI 辅助的多资源搜索问题”。

本文采用 ANF 作为语义基准：

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

其中 $\mathcal{M}$ 是需要综合的 square-free monomial 集合。该表示有两个优势。第一，任一候选线路都可

以回到 GF(2) 多项式进行等价验证。第二，搜索动作可以直接定义为对项集合的代数变换，例如公共因子提取、FPRM 极性重写、线性奇偶因子筛选和递归子问题分解。它的困难也很明确：原始 ANF 不显式给出共享结构，动作空间随项数和变量数迅速增长，必须用搜索策略、神经先验和保守兜底共同控制。

本文的一句话论证是：在逻辑层量子布尔函数 oracle 综合中，把 ANF/FPRM 项集合当作搜索状态，并用资源感知神经搜索选择可计算、可反算的代数分解动作，可以在传统函数和高维项集合上降低 T-count 与加权逻辑资源；其边界是当前结果仍停留在逻辑层，不包含 routing、magic-state factory、真实噪声模型或硬件后端映射。

本文贡献概括如下：

- 提出独立于 SSHR 的 Resource-NMCTS 搜索表述，以 ANF/FPRM 项集合为状态，以逻辑资源 score 为目标，把 Boolean oracle synthesis 写成可验证的组合搜索问题。
- 引入 Boolean-ring screen，在 square-free Boolean ring 中允许线性因子与 quotient 共享变量，从而扩大高维可搜索因子结构。
- 将 AI 模块分为动作排序、结构深度选择、depth-frontier policy 和 baseline-preserving guard，避免把学习策略写成不可控的黑箱生成器。
- 在 $n \leq 6$ 传统函数、 $n = 18, 20$ 函数级边界、 $n = 20$ 到 40 项集级 scale、 $n = 21, 22, 23$ 完整 truth-table bridge 以及 emitted-circuit schedule proxy 上给出资源、正确性和逻辑层时序证据。

2 相关工作与本文定位

2.1 布尔表示驱动的 oracle 综合

Xor-And-Inverter Graphs (XAG) 把 XOR 和 AND 结构分开，使 AND 节点数量与低 T-count oracle 编译直接相关。Meuli 等人讨论了 multiplicative complexity 在低 T-count oracle 编译中的作用 [1]，并将 XAG 用于量子编译 [2]。ROS 将 oracle synthesis 表述为 resource-constrained LUT mapping [3]，进一步说明布尔表示选择会影响 T-count、辅助比特和线路深度。BDD-based reversible synthesis 则展示了面向大函数的可扩展布尔表示路径 [4]。后端感知 oracle synthesis 开始把逻辑综合和 fault-tolerant back-end 指标连接起来 [5]。

本文与上述工作的共同点是承认布尔中间表示决定逻辑资源；差异在于本文不固定使用 XAG、LUT 或 BDD，而是在 ANF/FPRM 项集合上显式搜索可反算的因子结构。这样做的优点是语义验证直接，代价是候选动作空间更大，需要 AI 和 guard 控制搜索。

SSHR 利用 parallelotope 的空间结构做 small-scale Boolean function 的 CNOT-oriented synthesis [6]。本文不使用 SSHR 的候选空间，也不把论文目标设为 CNOT-only 最优。SSHR 在本文中是一个重要但定位明确的 baseline：它帮助说明本文在低 T-count 和加权 score 上的优势，也提醒我们在 CNOT 指标上必须诚实报告 trade-off。

2.2 学习式搜索与线路优化

学习式搜索已经出现在经典布尔电路设计、量子线路综合和 ZX-calculus 优化中。ShortCircuit 使用 AlphaZero 风格搜索进行经典布尔电路设计 [7]; Gumbel AlphaZero 被用于 Clifford+T unitary synthesis [8]; 强化学习和图神经网络也被用于 ZX-calculus rewrite-rule 选择 [9]; MonteQ 使用 MCTS 处理量子线路综合中的动作排序问题 [10]。这些工作证明, 离散线路综合中的动作选择可以受益于学习策略。

本文的区别在于状态不是任意量子门序列, 也不是 ZX 图, 而是 Boolean oracle 的 ANF/FPRM 项集合。神经模型不直接输出线路, 而是排序或选择候选代数分解动作; 最终线路仍由可验证的 compute/action/uncompute 结构生成。这一设计降低了 AI 生成错误线路的风险, 也使每个候选都可以经过 ANF 展开、GF(2) 符号模拟或完整 truth-table 检查。

3 问题定义与资源模型

本文输入是布尔函数的 truth-table 或 ANF 项集合, 输出是实现式 (1) 的逻辑层可逆线路。线路由 X、CNOT 和多控 Toffoli (MCT) 抽象门组成, 并在资源统计中估计 T-count、CNOT、depth、gate count 和 peak ancilla。搜索排序使用统一分数

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

该分数不是物理设备的时间或错误率模型, 而是逻辑层候选排序目标。为避免单一分数掩盖 trade-off, 实验同时报告各个资源分量。

正确性验证分三层。第一, 在函数级小规模和边界切片中构造完整 truth-table, 逐点检查 oracle 输出。第二, 在大规模项集实验中, 将 factor plan 展开回 ANF 项集合, 检查目标多项式是否等价。第三, 对 emitted X/CNOT/MCT circuit 做 GF(2) 多项式符号模拟, 检查输出线多项式、输入线复原和辅助线清零。第三层验证不等同于 2^n 个输入的完整枚举, 但它检查的是最终线路而不只是搜索 plan, 因而强于只报告资源估计。

表 1: 本文核心术语。

术语	本文含义
Resource-NMCTS	在 ANF/FPRM 项集合上进行资源加权搜索的主框架, 包括神经动作先验、MCTS、guard 和 portfolio 选择。
Boolean-ring screen	在 square-free Boolean ring 中搜索可复用线性因子的结构筛选分支, 允许线性因子与 quotient 共享变量。
Depth policy	在 single、depth-1 和 depth-2 screen 之间选择的结构级神经策略。
Depth-frontier policy	在 depth-2、depth-3 和 depth-4 screen 之间选择的高预算质量-时间折中策略。
Guard	以确定性 baseline 为兜底的保守门控, 只有当学习候选不劣于 baseline 时才接受。
Truth-table bridge	在 $n = 21, 22, 23$ 构造完整 truth-table, 把大规模项集符号验证和函数级完整验证连接起来。
SSHR	CNOT-oriented small-scale baseline; 本文方法不使用 SSHR parallelotope 候选。

4 方法

4.1 搜索状态、动作与线路生成

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的代数结构，将 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题，再递归生成 compute/action/uncompute 形式的线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子，以及由神经模型扩展的 root actions。

动作 a 被接受后，框架会估计其即时资源收益、子问题大小、辅助比特峰值和后续可分解性。神经动作先验对候选排序，MCTS 在有限预算内探索候选组合，rollout 使用确定性资源估计和已有 baseline 候选。最终 portfolio 在多个候选线路中按式 (3) 选取最优者。该过程的关键不是让 AI 直接创造量子门，而是让 AI 在大量可验证的代数动作中更早访问有资源收益的分支。

4.2 布尔环线性因子

传统线性因子筛选容易要求线性因子变量和 quotient 变量不相交。这个限制便于实现，但会排除 square-free Boolean ring 中合法的重叠结构。本文的 Boolean-ring screen 允许二者共享变量，并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (4)$$

线路层面仍可用 CNOT 计算线性奇偶量，再以该辅助量控制 quotient 子线路。这样，Boolean-ring screen 不是简单增加 beam 宽度，而是把原先被实现约束排除的代数候选重新放回可综合空间。

4.3 神经策略、结构策略与保守 guard

本文把 AI 的角色拆成三类。第一类是动作排序：模型根据候选覆盖率、项数变化、变量覆盖密度、次数分布和即时资源估计对候选动作打分。第二类是结构选择：depth policy 判断当前项集合适合 single、depth-1 还是 depth-2 Boolean-ring screen。第三类是 frontier 选择：depth-frontier policy 在 depth-2、depth-3 和 depth-4 Boolean-ring screen 之间选择，目标是在接近 depth-4 或 all-depth frontier 的质量时减少全深度枚举开销。

保守 guard 用来限制学习策略的风险。若学习候选相对确定性 baseline 出现 score 退化，则返回 baseline。Depth-2 skip guard 采用 zero-false-skip 阈值，优先避免把应该运行 depth-2 的项集合错误跳过。Screen-gate 则用于判断在某些边界函数上是否可以跳过昂贵的 Resource tail。这样的设计降低了策略模型分布外失误对资源结果的影响，但也意味着部分 AI 贡献表现为运行时间节省或质量-时间折中，而不是无条件的质量支配。

表 2: Resource-NMCTS 的模块与证据钩子。

模块	机制	主要证据类型
ANF/FPRM 搜索状态	把 oracle 综合写成项集合分解问题, 所有候选可回到 GF(2) 多项式验证。	小规模 truth-table 验证与项集级 ANF 验证。
Boolean-ring screen	搜索允许共享变量的线性因子, 递归处理 quotient/rest 子问题。	$n = 18, 20$ 函数级边界和 $n = 20-40$ scale。
神经动作先验与 MCTS	对候选动作排序并在有限预算内组合分解动作。	传统函数上的搜索消融和 portfolio 改善。
Depth policy / guard	学习 screen 深度或跳过较深搜索, 并用 baseline-preserving 规则兜底。	held-out 项集时间节省和无 score-loss guard。
Depth-frontier policy	学习高预算 depth-2/3/4 质量前沿的折中点。	$n = 20, 28, 40$ scale 与 $n = 21, 22, 23$ bridge。

5 实验设计

实验分为六个层次。第一层是 $n \leq 6$ 传统函数, 用 direct ANF、AND-direct ANF、ESOP cube beam、weighted ESOP-MILP、ABC/BDD 估计、ROS-style LUT proxy 和 SSHR-H/SSHR-I 作为 baseline, 主要回答小规模函数上是否形成低 T-count 和低加权资源优势。第二层是 $n = 14, 15, 16, 18$ 的高维外部导出集, 对 ABC-AIG/XAG/LUT/BDD 和 ROS-style LUT proxy 做统一 truth-table 校验, 主要回答外部布尔表示路线在高维上是否会削弱本文优势。第三层是 $n = 18, 20$ 函数级边界, 完整构造 truth-table 并验证 emitted oracle circuit, 主要回答高维函数上哪些搜索分支仍可运行。第四层是 $n = 20$ 到 40 项集级 scale, 不构造完整 truth-table, 而是对 plan 和 emitted circuit 做符号等价验证, 主要回答结构策略是否可扩展, 并通过独立 seed 的 $n = 24, 28, 32, 40$ 泛化集检查 frontier policy 的稳定性。第五层是 $n = 21, 22, 23$ truth-table bridge, 对一小组生成式 ANF 函数重新构造完整 truth-table, 专门检查项集级符号验证是否与函数级 oracle 验证一致。第六层是 emitted-circuit schedule proxy, 在不做硬件 mapping 的前提下估计并行逻辑深度、CNOT-depth proxy、T-depth proxy 和显式辅助线 lifetime area。

所有 paired comparison 只纳入函数名或项集名匹配、语义正确且未 timeout 的结果。Timeout 行保留为运行边界证据, 但不参与正确结果均值比较。所有结论限定在逻辑层资源, 不声称硬件 mapping、连通性约束或噪声模型下最优。

6 结果

6.1 传统函数上的低 T-count 与低 score 优势

表 3 给出 $n \leq 6$ 传统函数上的平均资源。相对 direct ANF, Pareto-Resource-NMCTS 平均 T-count 降低 72.25%, 平均 score 降低 67.80%。相对 ESOP cube beam, Pareto-Resource-NMCTS 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%。相对 weighted ESOP-MILP, 获得 167/3/7, 平均 score 降低 29.84%。SSHR-H 的平均 CNOT 最低, 因此本文不能主张 CNOT-only 支配; 更准确的结论是, 本文用一定 CNOT 与辅助比特 trade-off 换取更低 T-count 和更低加权

score。

表 3: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2

6.2 ROS-style LUT proxy 外部对比

官方 ROS 把 oracle synthesis 写成 resource-constrained LUT mapping, 并配合垃圾管理策略降低辅助线压力。当前环境尚不能直接运行官方 ROS/RevKit/mockturtle 流程, 因此本文新增一个明确标注的 ROS-style LUT proxy: 对同一导出 BLIF benchmark 运行 ABC ‘if -K’, 在 $K = 3, 4, 5$ 之间做 sweep; 每个映射后的 LUT 网络按 local ANF compute/action/uncompute 方式估计 X/CNOT/MCT 逻辑资源, 再按式 (3) 选择 best- K 。这不是官方 ROS 复现, 也没有 SAT garbage management, 但它比单个固定 $K = 4$ 的 ABC-LUT baseline 更强, 更适合作为 LUT 路线的压力测试。

该 proxy 覆盖 $n = 3-6$ 的 177 个传统函数, 以及 $n = 14, 15, 16, 18$ 的 132 个高维导出函数, 共 309 个函数。三档 K sweep 产生 927 个外部映射行, 全部通过 truth-table 检查, 没有 timeout、error 或 incorrect 行。Best- K proxy 相对固定 $K = 4$ 自身获得 219/0/90 的 score 胜/负/平, 平均 score 降低 18.12%; 因此它确实是比原 ABC-LUT 更强的 LUT baseline。在这个更强 baseline 上, Resource-NMCTS 仍在 309/309 个函数上取得更低 score, 平均 score 降低 83.77%; Pareto-Resource-NMCTS 为 309/0/0、平均 score 降低 84.27%。高维 profile variant 在 132 个高维函数上相对该 proxy 为 132/0/0、平均 score 降低 97.38%。

表 4: LUT proxy 外部对比。该 proxy 不是官方 ROS/RevKit/mockturtle 复现。

比较	指标	函数数	胜/负/平	平均 Δ
Resource-NMCTS vs ROS-style LUT proxy	score	309	309/0/0	-83.77%
Profile-Resource-NMCTS vs ROS-style LUT proxy	score	132	132/0/0	-97.38%
Pareto-Resource-NMCTS vs ROS-style LUT proxy	score	309	309/0/0	-84.27%
AND-direct ANF vs ROS-style LUT proxy	score	309	307/2/0	-67.45%
ROS-style LUT proxy vs fixed $K = 4$ LUT	score	309	219/0/90	-18.12%

6.3 高维函数级边界

在 $n = 18$ 的 12 个随机 ANF 函数上, adaptive depth-2 Boolean-ring screen 相对 single screen 获得 11/0/1, 平均 score 从 11349.06 降至 10670.94; 完整 Resource-NMCTS 平均 score 为 7315.62, 相对 adaptive screen 进一步降低 23.85%。这说明中等高维上 screen 是强候选生成器, 但不能完全替代 Resource tail。

在 $n = 20$ 边界函数上, FPRM root beam 和 fast linear-pair 均触发 300 s task timeout。Adaptive depth-2 screen 相对 single screen 获得 5/0/1, 平均 score 降低 7.47%; 集成该分支后, Resource-NMCTS 相对 AND-direct ANF 获得 5/0/1, 平均 score 降低 11.80%。该切片中完整 Resource tail 与 adaptive screen 资源持平, 因此 screen-gated Resource-NMCTS 可把平均时间从 77.10 s 降至 20.71 s。这个结果支持运行时门控, 但也提示 $n = 20$ 切片的主要质量收益来自结构筛选而非深层 MCTS tail。

表 5: $n = 18, 20$ 函数级边界结果。

切片	方法	平均 T	平均 CNOT	平均 score	平均时间 (s)
$n = 18$	Single screen	10389.33	16268.42	11349.06	0.82
$n = 18$	Adaptive depth-2 screen	9767.00	15301.58	10670.94	4.39
$n = 18$	Resource-NMCTS	6639.33	11380.92	7315.62	226.18
$n = 20$	AND-direct ANF	21516.67	33635.67	23491.15	10.41
$n = 20$	Single screen	20786.00	32492.50	22695.15	10.70
$n = 20$	Adaptive depth-2 screen	19535.33	30542.50	21331.24	20.67
$n = 20$	Resource-NMCTS	19535.33	30542.50	21331.24	77.10
$n = 20$	Screen-gated Resource-NMCTS	19535.33	30542.50	21331.24	20.71

6.4 结构级 AI 与高预算 frontier

Depth policy 在 $n = 14, 16, 18$ 项集上训练, 并在 held-out $n = 20$ 项集上评估。Policy 相对 single screen 获得 42/0/6, 平均 score 降低 6.57%; 相对 all-depth adaptive 平均 score 只高 0.28%, 但平均运行时间降低 34.71%。保守 depth-2 skip guard 在 held-out $n = 20$ test 中没有 false skip, 96 个样本全部与 fixed depth-2 screen score 持平, 并相对 all-depth adaptive 节省 24.74% 平均时间。

为判断 fixed depth-2 是否已经达到质量上限, 本文在 $n = 20, 28, 40$ 上运行 depth-3/4 Boolean-ring screen。Depth-4 相对 fixed depth-2 获得 49/0/23, 平均 score 降低 3.10%, 但运行时间增加 681.55%。因此, depth-3/4 是高预算质量模式, 而不是默认快速模式。Depth-frontier policy 将这一质量前沿学习化: 在 $n = 20, 28, 40$ scale harness 中, 它相对 fixed depth-2 获得 35/0/37, 平均 score 降低 2.19%; 相对 all-depth adaptive depth ≤ 4 平均 score 只高 0.97%, 但节省 58.69% 时间。独立 seed 的 $n = 24, 28, 32, 40$ 泛化集进一步显示, frontier policy 相对 fixed depth-2 获得 40/0/56, 平均 score 降低 1.85%; 相对 fixed depth-4 只高 0.61% score, 并节省 23.40% plan time。

表 6: 结构级策略与 depth-frontier 结果。

设置	比较	score 胜/负/平	平均 Δ score	平均 Δ time
held-out $n = 20$	depth policy vs single screen	42/0/6	-6.57%	+2328.66%
held-out $n = 20$	depth policy vs all-depth adaptive	0/6/42	+0.28%	-34.71%
held-out $n = 20$	depth-2 guard vs fixed depth-2	0/0/96	+0.00%	-0.54%
scale $n = 20, 28, 40$	screen depth-4 vs fixed depth-2	49/0/23	-3.10%	+681.55%
scale $n = 20, 28, 40$	frontier policy vs fixed depth-2	35/0/37	-2.19%	+503.20%
scale $n = 20, 28, 40$	frontier policy vs all-depth	0/16/56	+0.97%	-58.69%
generalization $n = 24, 28, 32, 40$	frontier policy vs fixed depth-2	40/0/56	-1.85%	+456.43%
generalization $n = 24, 28, 32, 40$	frontier policy vs depth-4	0/19/77	+0.61%	-23.40%

6.5 项集级 scale 与完整验证桥接

项集级 scale 覆盖 $n = 20, 22, 24, 28, 32, 36, 40$, 主 scale、extended scale、depth-frontier、frontier-policy rerun 和独立泛化 run 合计 4680 个方法行。所有方法行均通过两层验证: factor plan 的 ANF 展开验证为 4680/4680, emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证为 4680/4680, mismatch 总数为 0。该结果说明大规模项集资源表不是只统计 plan, 但仍需承认它不是完整 truth-table 枚举。

Truth-table bridge 进一步把完整验证推进到 $n = 21, 22, 23$ 。在 18 个生成式 ANF 函数和 10 个方法上, 180/180 个方法行通过完整 truth-table oracle 验证、plan ANF 展开验证和 emitted-circuit GF(2) 符号验证, mismatch 为 0。其中新增的 $n = 23$ bridge 单独包含 6 个函数、60 个方法行, 平均 truth-table 构造时间为 209.54 s/function。该 bridge 还复现了 frontier 信号: 在 $n = 21, 22$ 上, depth-frontier policy 相对 fixed depth-2 获得 8/0/4, 平均 score 降低 3.50%; 在 $n = 23$ 上获得 4/0/2, 平均 score 降低 1.88%。相对 all-depth adaptive, $n = 23$ frontier policy 平均 score 只高 0.61%, 但节省 48.77% plan time。

表 7: 大规模项集验证与 truth-table bridge。

验证项或比较	结果	说明
项集级 plan ANF 验证	4680/4680	覆盖 $n = 20$ 到 40 的 scale/frontier/frontier-policy rerun 和独立 seed 泛化 run。
项集级 emitted-circuit 符号验证	4680/4680	GF(2) 多项式模拟通过, mismatch 总数为 0。
$n = 21, 22, 23$ 完整 truth-table oracle 验证	180/180	18 个函数、10 个方法, 所有 emitted circuit 逐点验证通过。
$n = 21, 22$ frontier policy vs fixed depth-2	8/0/4	平均 score -3.50% , plan time $+634.71\%$ 。
$n = 23$ frontier policy vs fixed depth-2	4/0/2	平均 score -1.88% , plan time $+759.95\%$ 。
$n = 23$ frontier policy vs all-depth	0/1/5	平均 score $+0.61\%$, plan time -48.77% 。

6.6 逻辑层 schedule proxy 与辅助线生命周期

为避免只报告静态资源总量, 本文进一步在 emitted X/CNOT/MCT oracle circuit 上计算逻辑层 schedule proxy。该分析不做硬件 mapping, 也不使用连通性、routing、噪声或 magic-state factory 模型; 它只把每个 emitted gate 按线冲突做并行层调度, 并用 MCT 分解的 CNOT 数和 T-stage proxy 估计 CNOT-depth 与 T-depth, 同时记录显式辅助线从第一次使用到最后一次使用的 lifetime area。因此, 这组指标应理解为后端相关的逻辑层代理指标, 而不是物理设备运行时间。

表 8 显示, depth-frontier policy 的质量收益在 T-depth proxy 上也成立。在独立 seed 的 $n = 24, 28, 32, 40$ 泛化集中, frontier policy 相对 fixed depth-2 获得 40/0/56 的 score 胜/负/平, 平均 score 降低 1.85%; 同一比较下 T-depth proxy 也为 40/0/56, 平均降低 1.85%。在 $n = 21, 22$ 完整 truth-table bridge 中, frontier policy 相对 fixed depth-2 的 T-depth proxy 为 8/0/4、平均降低 3.32%; 在新增 $n = 23$ bridge 中为 4/0/2、平均降低 1.69%。代价也很清楚: 相对 fixed depth-2, 显式辅助线 lifetime area 在三组实验中分别增加 20.09%、32.93% 和 29.49%。相对 depth-4 或 all-depth frontier, frontier policy 只牺牲约 0.32%–0.55% 的 T-depth proxy, 在 $n = 23$ 上牺牲 0.52%, 同时减少约 4.01%–5.42% 的显式辅助线 lifetime area, 并显著减少搜索时间。

表 8: 逻辑层 schedule proxy 结果。所有指标来自 emitted circuit, 不包含硬件 mapping。

设置	对比	score	T-depth proxy	aux area
$n = 24, 28, 32, 40$ schedule 泛化	frontier policy vs fixed depth-2	40/0/56, -1.85%	40/0/56, -1.85%	+20.09%
$n = 24, 28, 32, 40$ schedule 泛化	frontier policy vs depth-4	0/19/77, +0.61%	0/19/77, +0.55%	-5.42%
$n = 21, 22$ schedule bridge	frontier policy vs fixed depth-2	8/0/4, -3.50%	8/0/4, -3.32%	+32.93%
$n = 21, 22$ schedule bridge	frontier policy vs depth-4	0/2/10, +0.32%	0/2/10, +0.32%	-4.01%
$n = 23$ schedule bridge	frontier policy vs fixed depth-2	4/0/2, -1.88%	4/0/2, -1.69%	+29.49%
$n = 23$ schedule bridge	frontier policy vs depth-4	0/1/5, +0.61%	0/1/5, +0.52%	-5.09%

7 证据地图与边界

表 9 将当前可写入论文的主张与证据对应起来。最重要的写法是把“AI”讲成可拆分、可验证的搜索控制模块，而不是泛泛声称使用强化学习；把“SSHR”讲成 CNOT-oriented baseline，而不是本文方法来源；把“高维结果”讲成项集级符号验证加 bridge，而不是所有 $n = 40$ 函数都做了完整 truth-table。

表 9: 当前主张、证据与投稿边界。

主张	当前证据	边界
本文路线独立于 SSHR	状态、动作和验证都定义在 ANF/FPRM 项集合上；SSHR 只出现在 baseline 比较中。	不能声称 CNOT-only 支配 SSHR。
低 T-count 与低加权 score 是主优势	$n \leq 6$ 传统函数相对 ESOP、weighted ESOP-MILP、direct ANF 和 AND-direct ANF 有稳定优势。	资源权重仍是逻辑层人为设定。
Boolean-ring screen 是高维有效结构筛选	$n = 18, 20$ 函数级切片和 $n = 20-40$ 项集级 scale 均显示相对 single screen 的 score 改善。	高维不是全局最优性证明。
结构级 AI 能降低搜索成本	Depth policy、skip guard、screen-gate 和 frontier policy 均有 held-out 或 scale 证据。	部分 AI 贡献是时间节省或质量-时间折中，不是绝对质量超过 oracle。
大规模结果具有语义可信度	4680/4680 项集级方法行通过 plan 和 emitted-circuit 符号验证；180/180 bridge 行通过完整 truth-table 验证。	$n = 24-40$ 仍未做完整 truth-table 枚举。
逻辑层后端相关 proxy 可解释 trade-off	schedule 泛化和 bridge 显示 frontier policy 同步降低 T-depth proxy，但相对 fixed depth-2 增加辅助线 lifetime area。	不是硬件 mapping、routing 或噪声模型结论。
投稿前仍需补强外部比较	当前已有 ESOP、ABC、BDD、SSHR、ROS-style LUT proxy 对比和 toolchain readiness 审计。	官方 ROS、mockturtle、RevKit 流程仍待复现。

8 讨论

本文最稳妥的贡献表述是“资源约束搜索”，不是“AI 全面战胜传统综合”。当前证据显示，ANF/FPRM 项集合提供了清晰的语义验证基础，Boolean-ring screen 提供了高维结构候选，MCTS 与神经先验提供了候选组合能力，depth policy 与 guard 控制搜索成本，depth-frontier policy 则把高预算质量前沿部分学习化。这些模块合在一起，形成了一条比单纯复现 SSHR 更适合投稿的逻辑层 Boolean oracle synthesis 主线。

同时，本文必须明确三个限制。第一，SSHR 的 CNOT-oriented 优势仍存在；在 CNOT-only 或 CNOT-depth profile 下，本文优势会收窄。第二，高维神经策略的独立质量收益仍不如结构筛选和 guard 稳定；当前最强高维证据来自 Boolean-ring screen 与 frontier policy 的结构选择。第三，schedule proxy 只是 emitted-circuit 层的逻辑调度代理，本文仍没有处理硬件 mapping、连通性、routing、magic-state factory 或噪声下可靠性，因而不能直接推出物理设备上更快或更可靠。

下一步最有价值的提升目标不是继续堆内部表格，而是补两个外部缺口。其一，在 ROS-style LUT proxy 之外复现或接入官方 ROS、mockturtle/RevKit 风格 reversible-toolchain，使比较从 proxy 扩展到更标准的综合流程。其二，把 frontier policy 的质量 gap 从当前相对 all-depth 或 depth-4 的约 0.32%–0.61% 进一步压低，同时保持接近 50%–60% 的时间节省，并减少相对 fixed depth-2 的辅助线 lifetime area 增幅。若这两点完成，论文主张会从“已有一条合理路线”提升到“有更强外部说服力的算法贡献”。

9 结论

本文提出 Resource-NMCTS，将量子布尔函数 oracle 综合建模为 ANF/FPRM 项集合上的资源约束搜索问题，并结合神经动作先验、MCTS、布尔环线性因子筛选、结构深度策略、depth-frontier policy 和保守 guard 生成逻辑层可逆线路。实验表明，该方法在 $n \leq 6$ 传统函数上显著降低 T-count 与加权 score，在 ROS-style LUT proxy 上保持 309/0/0 的 score 优势，在 $n = 18, 20$ 函数级边界上验证了 Boolean-ring screen 与 Resource tail 的互补作用，在 $n = 20$ 到 40 项集级 scale 中展示了结构级策略的可扩展性，并通过 $n = 21, 22, 23$ truth-table bridge 补强了高维正确性证据。新增 schedule proxy 进一步说明 frontier policy 的收益不仅体现在 score，也体现在 emitted-circuit T-depth proxy；同时它暴露了相对 fixed depth-2 的辅助线生命周期代价。当前稿件已经形成独立于 SSHR 的投稿主线，但最终论文仍需官方 ROS/RevKit/mockturtle 对比和真正后端相关评估来提升审稿说服力。

参考文献

- [1] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [2] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.

- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [7] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.
- [8] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [9] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [10] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.